AprilTag Height Anomaly Sorting

AprilTag Height Anomaly Sorting

1. Course Content
2. Preparation
3. Case Demonstration
4. Core Source Code Analysis

1. Course Content

This feature implements the program acquiring images through the camera and recognizing the AprilTags in the images, calculating the height of each AprilTag, and then removing AprilTags whose height is above 4 centimeters.

The wooden block used in this section: **30x30x30mm, 30x30x60mm AprilTag wooden blocks**.

2. Preparation

```
ros2 launch yahboomcar_bringup arm_demo.launch.py demo:=apriltag_remove_height
```

3. Case Demonstration

- Place the **30x30x30mm AprilTag wooden block** provided with the product within the camera's field of view



- Press the spacebar, and the robotic arm will lower its gripper to remove AprilTags with a height exceeding 4cm. The chassis will first align with the target. After the chassis alignment with the target is complete, grasping is performed. After grasping is complete, the block is placed in different positions according to the AprilTag number, and then the robotic arm resets.

4. Core Source Code Analysis

- Program path:

```
~/yahboomcar_ros2_ws/yahboomcar_ws/src/common_demo/common_demo/arm_demo/apriltag_remove_heig
ht.py
```

Here the flow of removing targets is explained:

_execute_remove_height function (removing height-anomalous AprilTags):

This function is the main control function of the removal flow. It first filters out the AprilTags whose height exceeds the threshold, and then performs the alignment, grasping, and placement operations for each target in turn.

Target filtering stage:

- `_get_all_tags_measure_pose(tags)` : obtains the 3D poses of all detected AprilTags in the measure

coordinate frame

- `pose[2] > self.remove_height`: filters AprilTags whose height (z coordinate) exceeds the removal threshold; `remove_height` defaults to 0.04m (4cm)
- `high_tags.sort(key=lambda item: item[0].tag_id)`: sorts by AprilTag ID to ensure a consistent processing order each time
- If there are no valid AprilTags, or no AprilTags exceeding the threshold, directly reset the state and return

One-by-one processing stage (for loop):

- `_iterative_align(tag_id)`: calls the iterative alignment function to adjust the chassis position to aim at the current AprilTag, and returns the aligned pose
- `_get_fresh_tag_data(tag_id)`: re-obtains the latest detection result after alignment is complete, and extracts the AprilTag's yaw angle (yaw)
- `normalize_angle_to_0_45(rvec_yaw_deg)`: normalizes the yaw angle to the range 0°~45°, used to determine the gripper rotation direction
- `grasp_offset`: grasping offset `[gx, gy, gz, pitch]`, compensating for the difference between the camera-detected position and the actual grasping position
- `arm_kinematics.inverse_kinematics(...)`: calls the inverse kinematics solver, taking the target position (camera pose + grasping offset) and pitch angle as input, and outputting 5 joint angles
- `result.approximate` / `result.adjusted_pitch`: handles the cases of approximate solutions and automatic pitch angle adjustment, outputting warning logs

Robotic arm motion and grasping stage:

- `joints[0:3] = round(ik_joints[0:3])`: rounds the J1~J4 joint angles obtained from the inverse solution to integers (the low-level firmware only accepts integers)
- `joints[4] = round(90 + angle_normalized)`: the J5 joint angle is calculated from the normalized yaw angle, so that the gripper rotation aligns with the AprilTag direction; 90° is the neutral position of the gripper
- `joints[5] = 30`: the J6 gripper angle is set to 30° (open state)
- `pubSix_Arm(joints)`: publishes the 6-joint control command, and the robotic arm moves to the grasping pose
- `SingleJoint_pub.publish(ArmJoint(id=6, joint=gripping_angle, time=1500))`: controls the gripper to close individually
- `down_joint = [180, 35, 70, 5, 60, gripping_angle]`: the preset placement joint angles, moving the AprilTag to the side of the robot
- `SingleJoint_pub.publish(ArmJoint(id=6, joint=30, time=1500))`: opens the gripper to release the object
- `pubSix_Arm(observation_pose)`: returns the robotic arm to the observation pose to prepare for processing the next AprilTag

```
    def _execute_remove_height(self, tags):
        """Main picking-thread logic: process all tags whose height exceeds remove_height
one by one"""
```

```
        self.get_logger().info(f"Remove height task started, threshold=
{self.remove_height:.3f}m")

        # Get the measure poses of all tags and filter out those with excessive height
        tags_with_pose = self._get_all_tags_measure_pose(tags)
        if not tags_with_pose:
            self.get_logger().info("No valid tags detected")
            self._reset_grasp_state()
            return

        high_tags = [(tag, pose) for tag, pose in tags_with_pose if pose[2] >
self.remove_height]
        if not high_tags:
            self.get_logger().info(f"No tags exceed remove_height
({self.remove_height:.3f}m)")
            self._reset_grasp_state()
            return

        high_tags.sort(key=lambda item: item[0].tag_id)
        self.get_logger().info(
            f"Found {len(high_tags)} tag(s) exceeding remove_height: "
            f"{[t.tag_id for t, _ in high_tags]}, heights: {[f'{p[2]:.3f}m' for _, p in
high_tags]}"
        )

        for tag, init_pose in high_tags:
            if not rclpy.ok():
                break
            tag_id = tag.tag_id
            self.get_logger().info(f"--- Processing tag ID {tag_id}, height=
{init_pose[2]:.3f}m ---")

            # Iterative chassis alignment
            final_pose = self._iterative_align(tag_id)
            if final_pose is None:
                self.get_logger().error(f"Alignment failed for tag {tag_id}, skipping")
                continue

            time.sleep(0.5)
            latest_tag, _, rvec_yaw_deg = self._get_fresh_tag_data(tag_id)
            if latest_tag is None:
                self.get_logger().error(f"Tag {tag_id} lost after alignment, skipping")
                continue

            angle_normalized = normalize_angle_to_0_45(rvec_yaw_deg)
            self.get_logger().info(f"Tag {tag_id} rvec yaw: {rvec_yaw_deg:.1f}° ->
normalized: {angle_normalized:.1f}°")

            # Apply grasp_offset to the IK target position
            mx, my, mz = final_pose
            gx, gy, gz = self.grasp_offset[0], self.grasp_offset[1], self.grasp_offset[2]
            self.get_logger().info(
                f"Tag {tag_id} measure pose: x={mx:.3f}, y={my:.3f}, z={mz:.3f}; "
```

```
                f"IK target with grasp_offset: x={mx+gx:.3f}, y={my+gy:.3f}, z={mz+gz:.3f},
pitch={self.grasp_offset[3]:.1f}°"
            )

            result: KinResult = self.arm_kinematics.inverse_kinematics(
                target_x=mx + gx,
                target_y=my + gy,
                target_z=mz + gz,
                target_pitch=self.grasp_offset[3],
                seed_angles_deg=None,
            )

            if not result.success:
                self.get_logger().error(f"IK failed for tag {tag_id}: {result.message}")
                continue

            if result.approximate:
                self.get_logger().warn(Fore.YELLOW + f"IK approximate solution, position
error = {result.position_error:.4f} m" + Fore.RESET)
            if result.adjusted_pitch is not None:
                self.get_logger().warn(Fore.YELLOW + f"IK pitch sweep, adjusted pitch =
{result.adjusted_pitch:.1f}°" + Fore.RESET)

            ik_joints = result.data
            joints = [0, 0, 0, 0, 0, 0]
            joints[0] = round(ik_joints[0])
            joints[1] = round(ik_joints[1])
            joints[2] = round(ik_joints[2])
            joints[3] = round(ik_joints[3])
            joints[4] = round(90 + angle_normalized) # 90 is the neutral position of the
gripper servo
            joints[5] = 30

            self.get_logger().info(f"Tag {tag_id} grasp joints: {joints}")
            self.pubSix_Arm(joints)
            time.sleep(3.5)
            self.SingleJoint_pub.publish(ArmJoint(id=6, joint=self.gripping_angle,
time=1500))
            time.sleep(2.0)

            down_joint = [180, 35, 70, 5, 60, self.gripping_angle]
            self.pubSix_Arm(down_joint)
            time.sleep(2.5)
            self.SingleJoint_pub.publish(ArmJoint(id=6, joint=30, time=1500))
            time.sleep(2.0)

            # Return to the observation pose and get ready for the next tag
            self.pubSix_Arm(self.observation_pose)
            time.sleep(2.5)
```